# CSE 331: Computer Security Fundamentals

Spring 2026

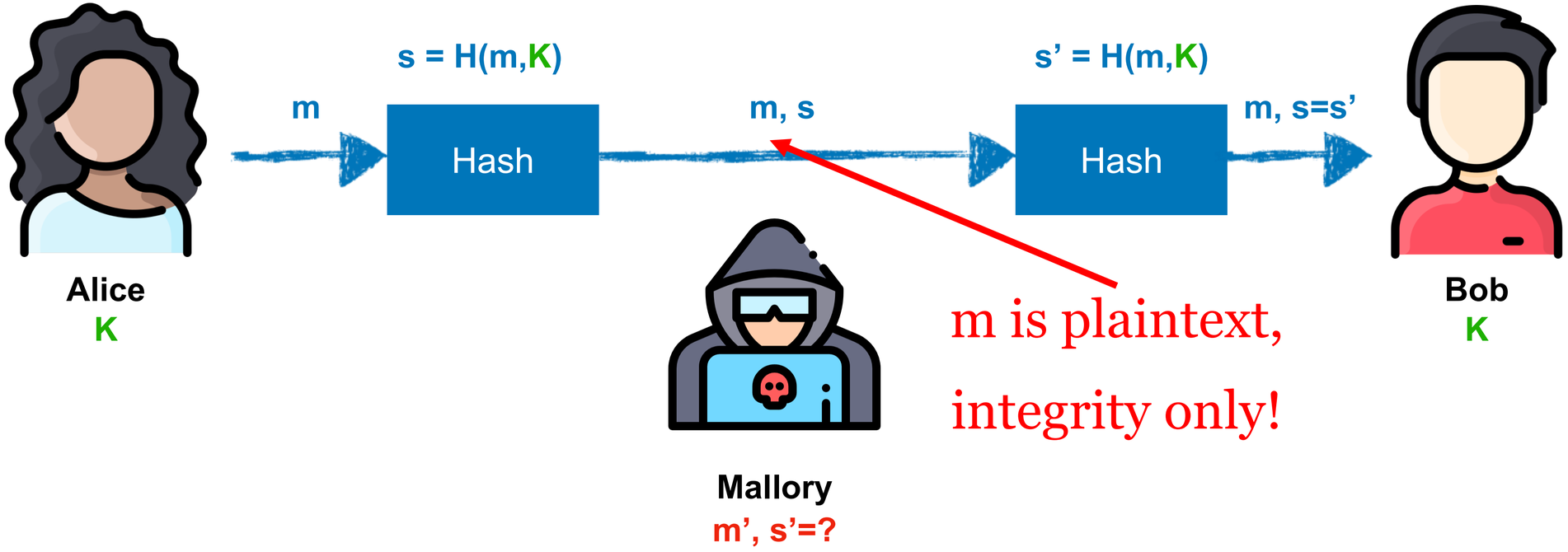
Basic Cryptography:
MACs and Hashes

Amir Rahmati
Xinrui Yu

Stony Brook Institute
at Anhui University
安徽大学纽约石溪学院

# How To Achieve Integrity?

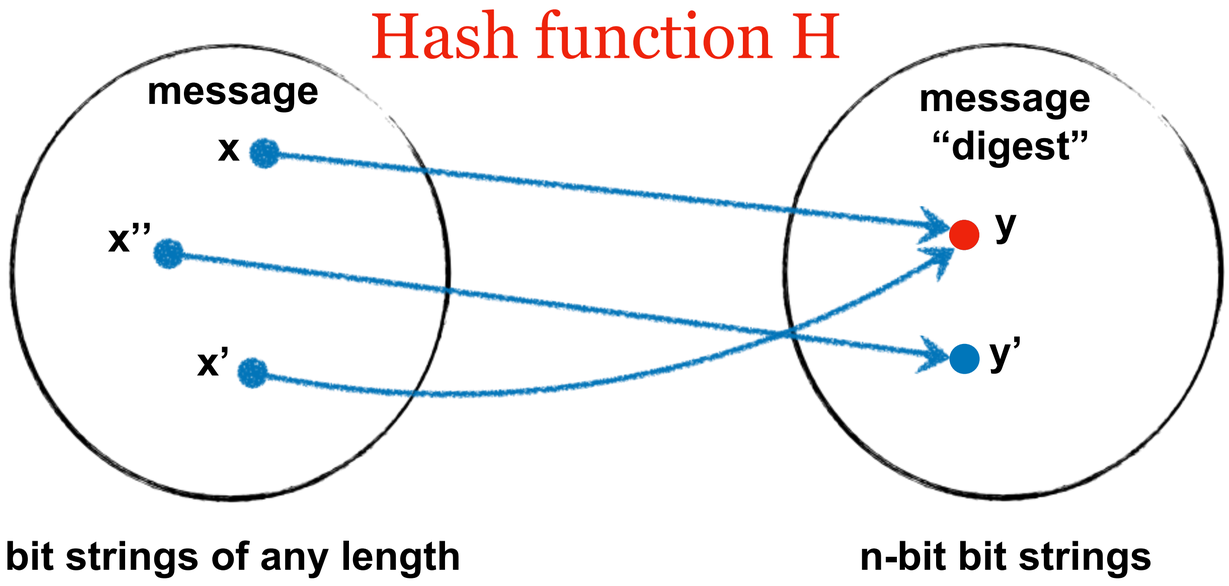
Message Authentication Schemes/Code (MAC)




I don't care who listens—I just need you to know the words are mine.

# Hash Functions

# Hash Functions: Main Idea



- Hash function H is a lossy compression function
- Collision: h(x)=h(x') for distinct inputs x, x'
- H(x) should look "random"
  - Every bit (almost) equally likely to be 0 or 1

# Hashing vs. Encryption

- Hashing is one-way. There is no "un-hashing"
  - A ciphertext can be decrypted with a decryption key… hashes have no equivalent of "decryption"
- Hash(x) looks "random" but can be compared for equality with Hash(x')
  - Hash the same input twice → same hash value
  - Encrypt the same input twice → different ciphertexts (CBC, CTR)
- Cryptographic hashes are also known as "cryptographic checksums" or "message digests"
- Cryptographic hash function needs a few properties…

# Property 1: One-Way

- Intuition: hash should be hard to invert
  - "Preimage resistance"
  - Given y, it should be hard to find any x such that h(x)=y
- How hard?
  - Brute-force: try every possible x, see if h(x)=y
  - SHA-1 (Secure Hash Algorithm) has 160-bit output
    - Expect to try $2^{159}+1$ inputs before finding one that hashes to y.

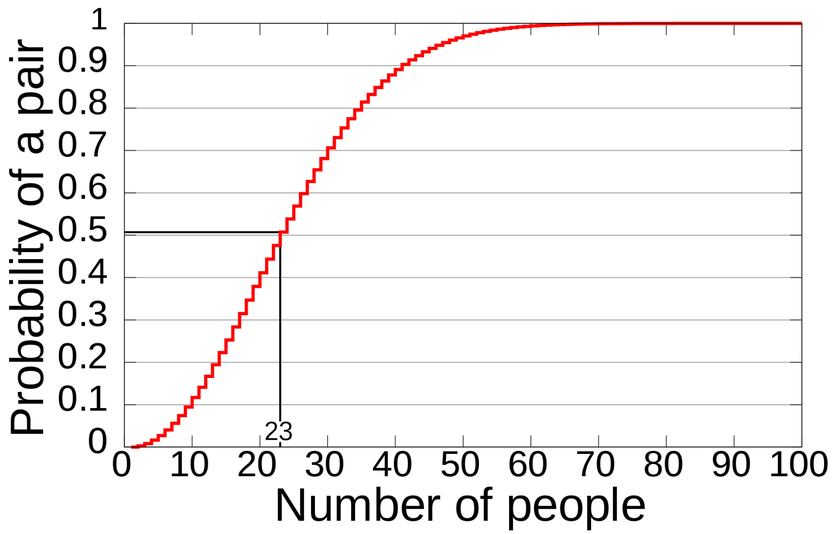
# Property 2: Collision Resistance

- Should be hard to find $x \neq x'$ such that $h(x)=h(x')$

# Birthday Paradox

- In this class of 50, what is the probability that two of you share the same birthday?
  - 365 days in a year (366 some years)
    - P(atleast 1 shared birthday) = 1 – P(all unique birthdays)
    - $P(Nunique) = \frac{365 \times 364 \times 363 \times \ldots}{365^N}$
    - Expect birthday "collision" with a room of only 23 people (50%).
    - For simplicity, approximate when we expect a collision as $\sqrt{365}$
- Why is this important for cryptography?
  - $2^{128}$ different 128-bit values
    - Pick one value at random. To exhaustively search for this value requires trying on average $2^{127}$ values.
    - Expect "collision" after selecting approximately $2^{64}$ random values.
    - 64 bits of security against collision attacks, not 128 bits.

**> 97% !!**



# Property 2: Collision Resistance

- Should be hard to find any $x \neq x'$ such that $h(x)=h(x')$
- Birthday paradox means that brute-force collision search is only $O(2^{n/2})$, *not* $O(2^n)$
  - For SHA-1, this means $O(2^{80})$ vs. $O(2^{160})$

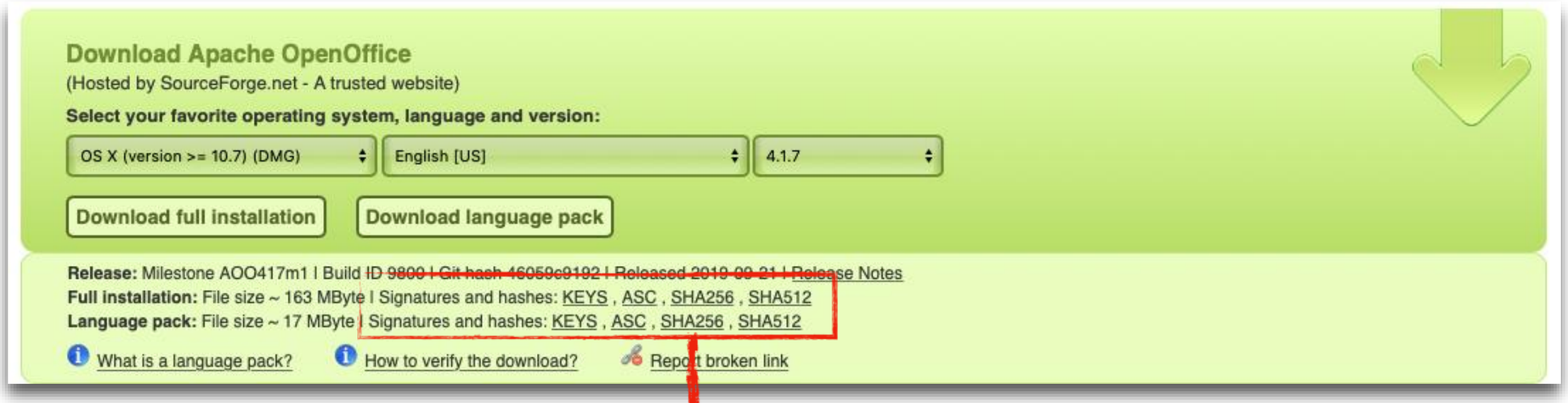
# Property 3: Weak Collision Resistance

- Given randomly chosen x, hard to find x' such that h(x)=h(x')
  - Attacker must find collision for a <u>specific</u> x. By contrast, to break collision resistance it is enough to find <u>any</u> collision.
  - Brute-force attack requires $O(2^n)$ time
  - "second-preimage resistant" (finding a second object x')
- Collision resistance => Weak collision resistance

# Application: Password Hashing

- Instead of user password, store hash(password)
- When user enters a password, compute its hash and compare with the entry in the password file
- Why is hashing better than encryption here?
    - System does not store actual passwords!
    - Cannot go from hash to password!

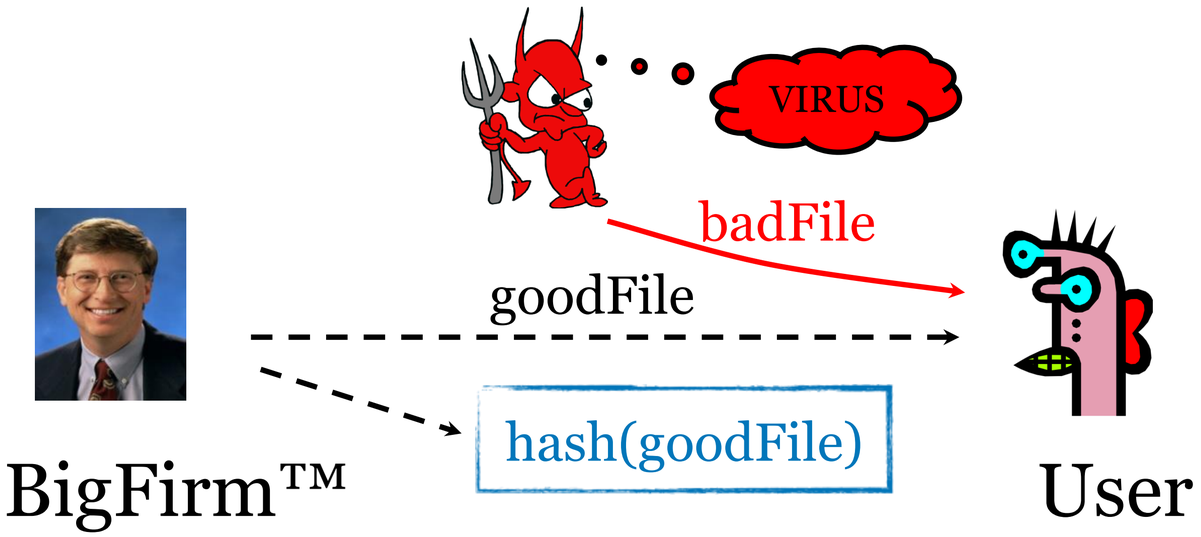
# Application: Software Integrity




```
615d6fa38ef78e99e30f515ce3f61fd9e228408a425c91afc5e587d662e18f99 *Apache_OpenOffice_4.1.7_MacOS_x86-64_install_en-US.dmg
```

```
[rahmati@Ameers-iMac Downloads % openssl dgst -sha256 Apache_OpenOffice_4.1.7_MacOS_x86-64_install_en-US.dmg
SHA256(Apache_OpenOffice_4.1.7_MacOS_x86-64_install_en-US.dmg)= 615d6fa38ef78e99e30f515ce3f61fd9e228408a425c91afc5e587d662e18f99
```

# Application: Software Integrity




**Goal:** Software manufacturer wants to ensure file is received by users without modification.

**Idea:** given goodFile and hash(goodFile), very hard to find badFile such that hash(goodFile)=hash(badFile)

# Application: Secret Auction

- Submit H(bid) instead of the bid.
- Reveal bid after deadline is over. Auctioneer checks hashes to make sure no one has changed their bid value.

# Which Property Do We Need?

## One-wayness, Collision Resistance, Weak CR?

- **Password Storage**
  - One-wayness: hard to recover the/a valid password
- **Integrity of software distribution**
  - Weak collision resistance
  - But software images are not really random… may need full collision resistance if considering malicious developers
- **Private auction bidding**
  - One-wayness: rival bidders should not recover B (this may mean that she needs to hash some randomness with B too)
  - Collision resistance: Alice should not be able to change her mind to bid B' such that H(B)=H(B')

# Common Hash Functions

In 2004, Xiaoyun Wang and co-authors demonstrated a collision attack against MD5.

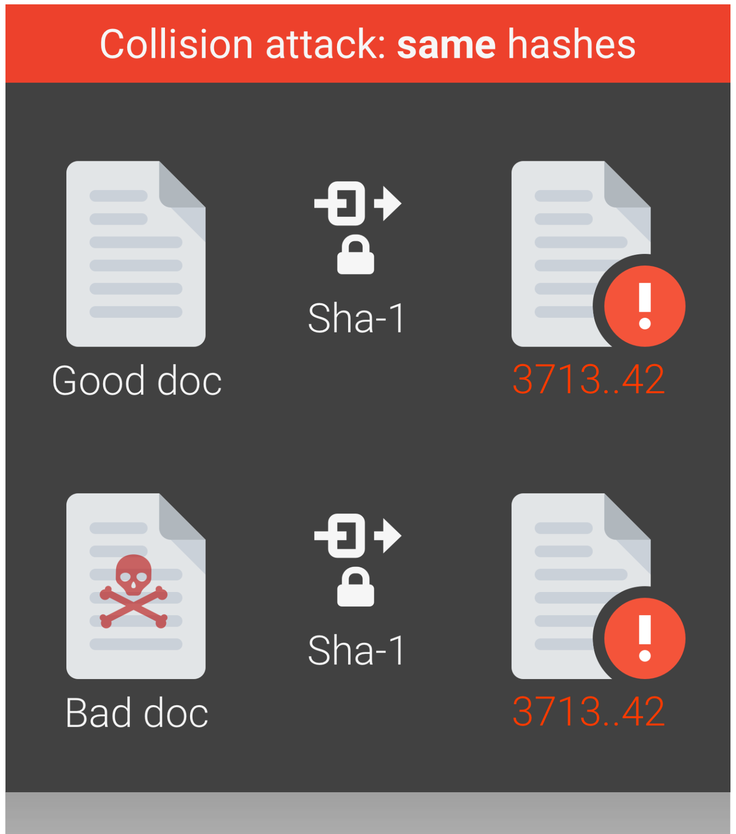
- **MD5**
  - 128-bit output
  - Designed by Ron Rivest, used very widely
  - Collision-resistance broken (summer of 2004): **MD5 considered harmful Someday**
  - Practical attack 2008: **MD5 considered harmful today**
- **RIPEMD-160**
  - 160-bit variant of MD5
- **SHA-1** (Secure Hash Algorithm)
  - 160-bit output
  - US government (NIST) standard as of 1993-95
  - Theoretically broken 2005; practical attack 2017!
- **SHA-2: Common today, includes SHA-256, SHA-512, SHA-224, SHA-384**
- **SHA-3**: standard released by NIST in August 2015

# SHA-1 Broken in Practice (2017)

**Google just cracked one of the building blocks of web encryption (but don't worry)**

*It's all over for SHA-1*

by Russell Brandom | @russellbrandom | Feb 23, 2017, 11:49am EST



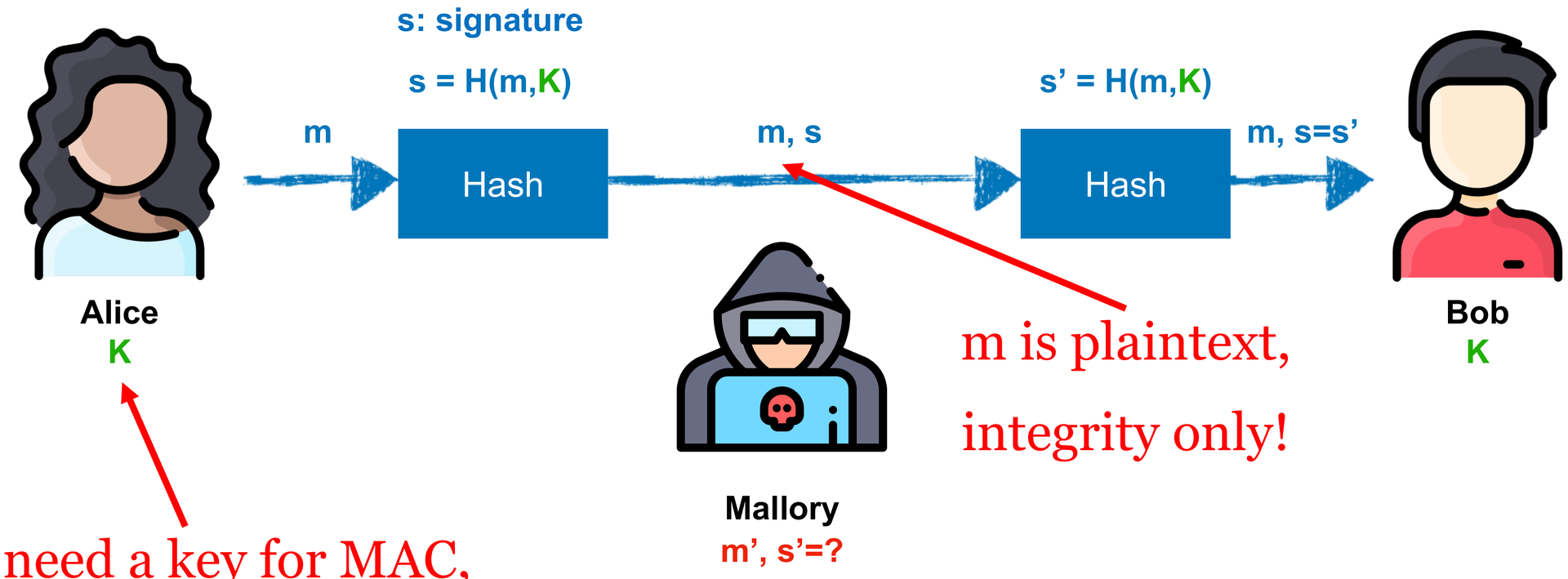

- Every bit of output depends on every bit of input
  - Important property for collision resistance
- Brute-force inversion: 2^80 (birthday attack)
- Weakness in 2005: collisions can be found in 2^63 operations

https://shattered.io

# Recap: To Achieve Integrity

## Message Authentication Schemes/Code (MAC)

Not Media Access Control address (MAC Address)



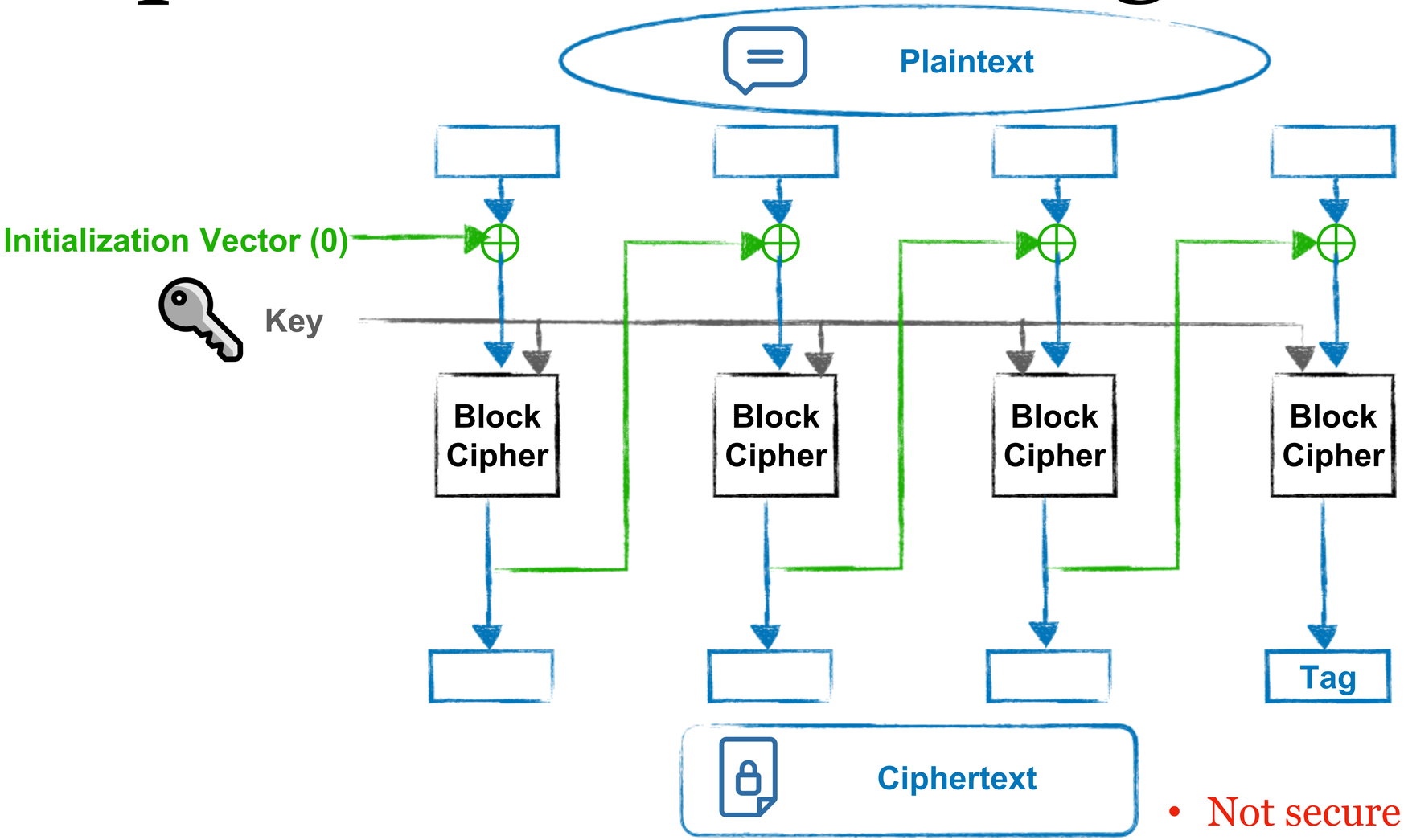

m is plaintext, integrity only!

You need a key for MAC, not needed for hash

# When is a MAC secure?

- MAC should be unforgeable
  - Hard to generate a valid (m’, s’) pair without knowing the key
  - Even when the attacker has many other (m, s) pairs

# Cipher Block Chaining (CBC) MAC




$$C_i = E_K(P_i \oplus C_{i-1})$$

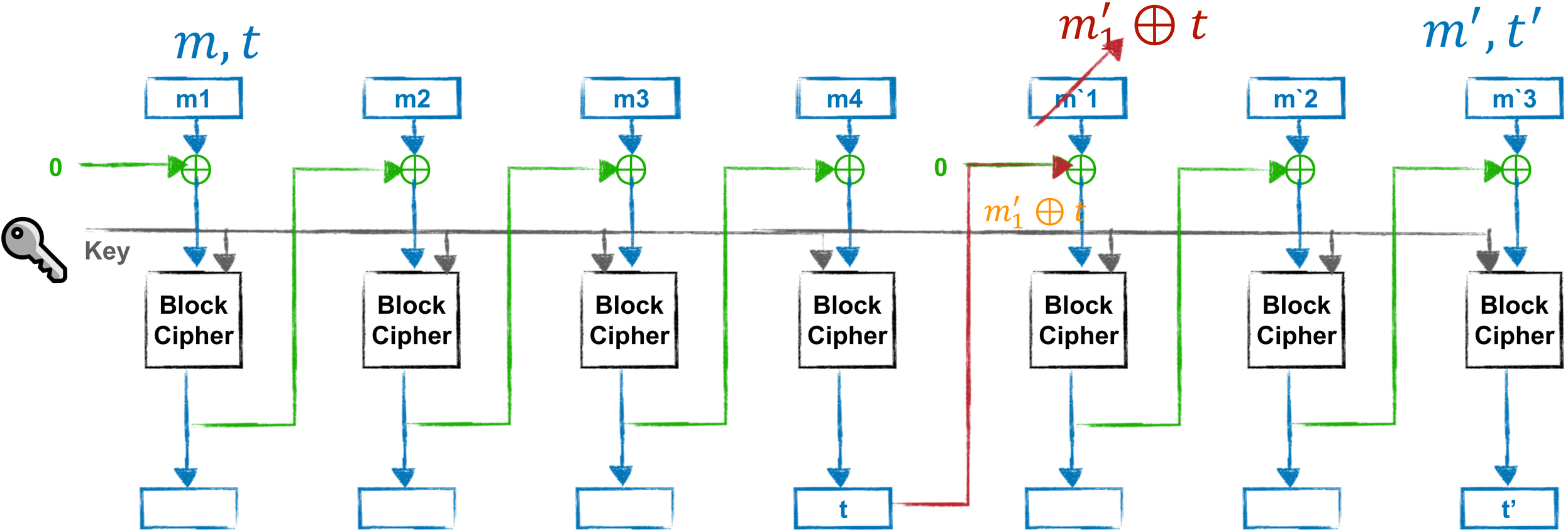
**The final $C_i$ becomes the tag.**

- Not secure when system may generate MAC for messages of different lengths.

# CBC-MAC Example

- **Sender (Alice) Computes the Tag:**
  1. Alice has a message M.
  2. She splits it into blocks and applies E in CBC mode **without an IV**.
  3. The final block's ciphertext is used as the **authentication tag**.
  4. She sends (M, Tag) to Bob.
- **Receiver (Bob) Verifies the Tag:**
  1. Bob recomputes the CBC-MAC on M using the shared key.
  2. If Tag matches his computed tag, the message is **authentic**. (Integrity)
  3. If not, he rejects it as potentially **tampered**. (Integrity Lost)

# Length Extension Attack



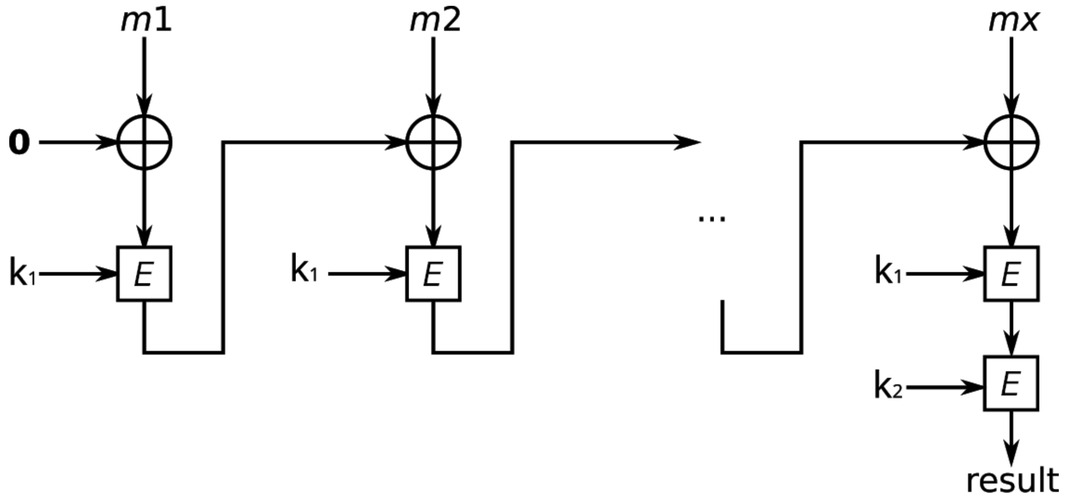

$m'', t''$

$$m'' = m \parallel [(m_1' \oplus t) \parallel m_2' \parallel \cdots \parallel m_x']$$
$$t'' = t'$$

"||" identifies the appending of one string onto another

# Solutions?

- Length Prepending (telling the length beforehand)
- Encrypt-last-block: $CBC-MAC-ELB(m,(k1,k2)) = E(k2, CBC-MAC(k1,m))$



# HMAC

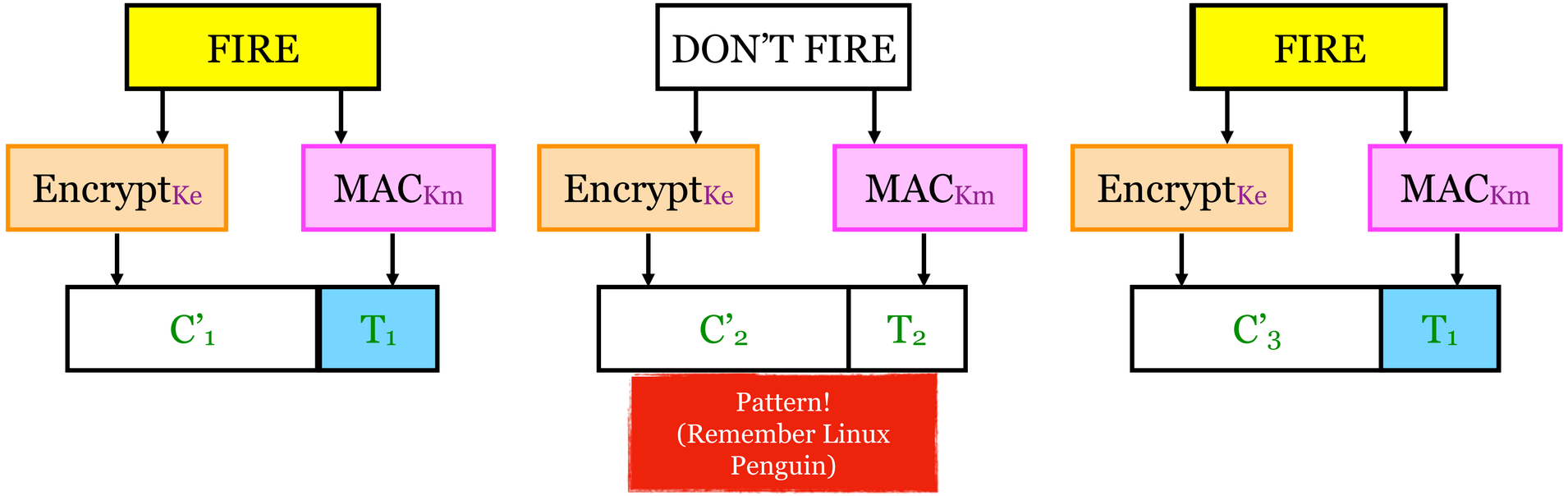
- Construct MAC from a cryptographic hash function
  - Invented by Bellare, Canetti, and Krawczyk (1996)
  - Used in SSL/TLS, mandatory for IPsec
- Why not encryption?
  - Hashing is faster than block ciphers in software
  - Can easily replace one hash function with another
  - There used to be US export restrictions on encryption

$$HMAC(K, m) = h((K \oplus a)||h((K \oplus b)||m))$$

**a and b are predefined constants**

# Authenticated Encryption

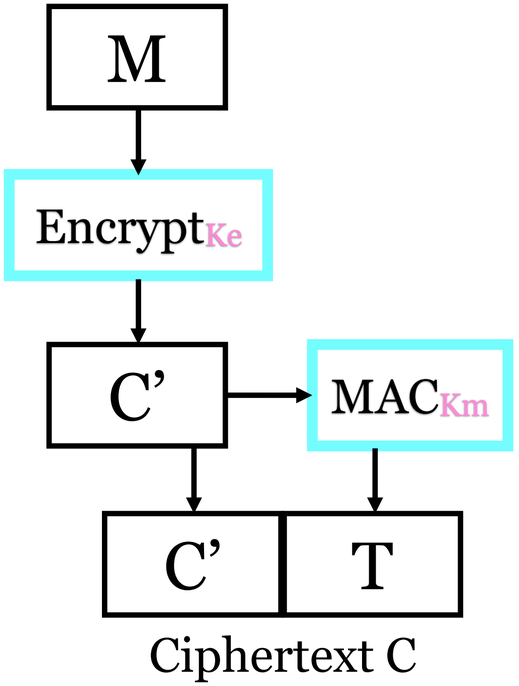
- What if we want <u>both</u> confidentiality and integrity?
- Natural approach: combine encryption scheme and a MAC.
- Obvious approach: Encrypt-and-MAC
- Problem: MAC is deterministic! same plaintext →same MAC



# Authenticated Encryption

- Instead: Encrypt *then* MAC.

- MAC-then-Encrypt
  - Not as good (need decrypt before verification)




**Encrypt-then-MAC**

# Office Hours

Lixi 201-4

- Monday 09:30 – 11:00 11:30 – 12:30
- Tuesday 11:30 – 12:30
- Thursday 11:30 – 12:30
- Friday 09:30 – 11:00

# Example Questions

Which is correct regarding the difference between MAC and hash functions?

A. MAC is bi-directional; hash is one-way.

B. MAC is used for confidentiality; hash is used for integrity.

C. MAC needs a key; hash does not need a key.

D. MAC can be verified; hash cannot be verified.

# Example Questions

Hash function is useful for:

1. Password storage/password hashing
2. Software integrity/download integrity
3. Secret auction

# Example Questions

K1, K2 are keys unknow to the attacker, M is plaintext message, and C is sent over the channel.

Does this scheme guarantee:

1) Confidentiality No, EBC leaks statistical information (remember Linux penguin)

2) Integrity No, CBC-MAC is vulnerable to length extension attack

C1 = ECB(K1, M) C2 = CBC-MAC(K2||C1) C = (C1||C2)